2024 IEEE International Conference on Image Processing (ICIP) | 979-8-3503-4939-9/24/$31.00 ©2024 IEEE | DOI: 10.1109/ICIP51287.2024.10647638

FEDAWA: AGGREGATION WEIGHT ADJUSTMENT IN FEDERATED DOMAIN GENERALIZATION

Yiming Chen *Nan He* *Lifeng Sun*⋆

Department of Computer Science and Technology, Tsinghua University
chenym22@mails.tsinghua.edu.cn henancs@mail.tsinghua.edu.cn sunlf@tsinghua.edu.cn

ABSTRACT

Domain Generalization (DG) in the setting of federated learning (i.e. Federated Domain Generalization, FDG) is gaining increasing attention. FDG aims to learn a global model generalizing well to clients with unseen domains in a privacy-preserving manner. Existing works mainly focus on how to learn invariant features or transmit data distributions in a privacy-preserving manner, the correctness of aggregation weights has not been extensively discussed. In this paper, we discuss the rationality of aggregation weights and propose a novel method in FDG, namely FedAWA. We replace traditional weighted aggregation based on data volume with arithmetic mean. Then we propose confidence adjustment to adjust aggregation weights to ensure the rationality of aggregation weights due to the uncertainty of local model training. Moreover, we propose fairness adjustment to adjust local training epochs to prevent local models from contributing excessively or insufficiently to the global model. Our method can serve as a plug-and-play plugina. Experiments on several benchmarks have shown the effectiveness of our method, our method enhances the generalization improvement of federated learning algorithms.

***Index Terms*—** Federated Learning, Federated Domain Generalization, Data Heterogeneity, Aggregation Weights Adjustment

1. INTRODUCTION

With the development of deep learning, large amounts of data are used for model training. But in many real-world scenarios, such as healthcare and finance, data are highly sensitive and not allowed to be shared. Moreover, policies such as the General Data Protection Regulation (GDPR) [1] stipulate rules on data sharing among different organizations. Thus federated learning (FL) has emerged as a promising machine learning paradigm [2], which has good predictive accuracy while obeying policies and regulations to protect privacy.

Existing works in FL mainly attempt to improve the performance of the global model on data belonging to the same domain as the training data, while test data from different domains is more practical. Thus it is necessary to investigate the problem of how to train a global model in FL that performs optimally on an unseen domain, i.e. Federated Domain Generalization (FDG) [3]. However, FDG has not been fully explored yet. The principal challenge of FDG is how to address the issue of domain shift between training and test domains. FDG is similar to Domain Generalization (DG) in machine learning, both aiming to generalize from multi-source domains to unseen domains. The key difference of two problems is that it is unacceptable to share data in FDG, and it brings many new challenges.

Currently, methods in FDG focus on representing distribution information through certain cues (e.g, amplitude spectrum [4]) or learning an unbiased model across domains [5]. Some works also aim to enhance performance on unknown domains by optimizing for generalization gap [6], which is achieved by adjusting the aggregation weights. These works are built upon traditional FL settings, inheriting the conventional FL approach of using aggregation weights weighted by the data volume. However, it seems that these works overlook a crucial point: the method of aggregation weights based on data volume should not be used because there is no such thing that the larger the amount of data, the closer the trained local model will be to the optimal global model in FDG.

Therefore we propose a new method to solve the above problems. Our method first reexamines the aggregation weights in FDG and replaces the traditional weighted average based on data volume with arithmetic mean. Then, we believe that in practice, due to different stages of training and factors such as training difficulty, there is a certain gap between the local model and the optimal local model. Therefore, we fine-tune the aggregation weights, and we propose the confidence adjustment to achieve this. Furthermore, we propose the fairness adjustment of local training epochs to ensure that no client contributes excessively or insufficiently to the global model. Our contributions are summarized below:

- We identify shortcomings in the aggregation weight of existing works and introduce a new method FedAWA that replaces the weighted averages based on data volume with arithmetic mean.

⋆Corresponding author

Authorized licensed use limited to: IEEE Xplore. Downloaded on September 29,2024 at 14:05:39 UTC from IEEE Xplore. Restrictions apply.

- Confidence adjustment of aggregation weights and fairness adjustment of local training epochs are proposed to further ensure the rationality and fairness of aggregation weights.
- We test our method FedAWA in FL setting on two standard DG benchmarks (PACS and Office-Home) and the results prove that FedAWA brings stable and significant performance improvements when used as a plugin with other FL methods.

2. RELATED WORK

2.1. Domain Generalization

Domain Generalization is a machine learning problem, it considers the scenarios where target data is inaccessible during model learning [7]. The existing methods in DG can be categorized into three groups [8], including manipulating the inputs to assist learning general representations [9, 10, 11], feature disentanglement [12] or domain-invariant representation learning [13], and exploiting learning strategy to promote the generalization capability [14], such as ensemble learning, meta learning, self-supervised learning and so on.

2.2. Federated Domain Generalization

The aim of federated learning is to train machine learning models on private data across a network of IoT devices [15]. Most existing works focus on solving the issue of heterogeneity (data, device, and model, etc.) [15, 16] or accelerate the speed of convergence [17, 18] in FL, FedAvg [15] is one of the most representative methods which performs weighted parameter average over the client models after several local updates. And there are many methods to reduce potential weight divergence or reduce communication rounds by using regularization term to the local object, such as FedProx [16], FedMMD [19] and FedDyn [20]. In addition, some works propose normalized averaging method to eliminate objective inconsistency, such as FedNova [21]. Currently, most existing methods only consider the imbalance of labels and ignore the imbalance of domains, while the latter is more in line with the reality.

There are some works addressing the DG problem in FL. FDG [4] utilizes the amplitude spectrum as distribution information and exchanges them cross domains to relieve domain shift, CCST [22] utilizes cross-client style transfer to introduce all the source client styles into each client. However, the two methods pose significant risks of data leakage. FedSR [5] aims to learn a simple representation of the data for better generalization by introducing two regularizes. FedDG-GA [6] reduces the variance of generalization gaps by dynamically calibrating the aggregation weights, our method calibrates the aggregation weights too but for another purpose.

3. METHOD

In this section, we introduce our proposed method FedAWA: Aggregation Weight Adjustment in Federated domain generalization, which is shown in Figure 1. We introduce FedAWA in three parts: the motivation of FedAWA (Sec. 3.1), confidence adjustment of aggregation weights (Sec. 3.2) and fairness adjustment of local training epochs (Sec. 3.3).

3.1. Motivation

Suppose that there are K clients, and each client i $(1 \leq i \leq K)$ has its own dataset with N_i samples: $D_i = \{(x_i^{(1)}, y_i^{(1)}), ..., (x_i^{(N_i)}, y_i^{(N_i)})\}$, and the data of each client i comes from a unique domain. We notate θ_i as the model parameters from i^{th} client and θ_g as the global model parameters generated by the server. And the server aggregates a global model through the locals models uploaded by clients, which can be formulated as:

$$\theta_g = \sum_{i}^{K} w_i \theta_i, \text{where } \theta_i = \underset{\theta_i}{\operatorname{argmin}}\, L_i(\theta_i) \tag{1}$$

where w_i represents the aggregation weight of client i in global model, usually determined as the volume of local samples, i.e. $w_i = N_i / \sum_{i'}^{K} N_{i'}$. Currently, most of the existing works in FDG utilize this aggregation weight. $L_i(\theta_i)$ represents the loss value of client i.

The aggregation weights are represented by the proportion of samples in conventional federated learning. This practice stems from the belief that as the number of samples increases, the local model becomes closer to the optimal global model, so the weights in the aggregation process should be proportionally higher. However, this phenomenon does not exist in Federated Domain Generalization. In FDG, we assume that the optimal local model θ^* corresponding to a randomly chosen domain follows a Gaussian distribution, i.e. $\theta^* \sim \mathcal{N}(\mu_\theta, \sigma_\theta)$, and setting the global model to μ_θ is the most reasonable.

We denote the optimal local models corresponding to K clients as $\{\theta_1^*, \theta_2^*, ..., \theta_K^*\}$. Assuming θ_i^* is an independent observation from the Gaussian distribution $\mathcal{N}$, the parameters of the Gaussian distribution μ_θ can be estimated through maximum likelihood estimation (MLE). By maximizing the likelihood function $ln(\{\theta_1^*, \theta_2^*, ..., \theta_K^*\}|\mu_\theta, \sigma_\theta)$, the optimal global model θ_g^* can be solved as:

$$\theta_g^* = \mu_\theta = \frac{1}{K} \sum_{i}^{K} \theta_i^* \tag{2}$$

Therefore, under our setting, the aggregation process in the server should apply the arithmetic mean of total local models rather than the weighted average based on data volume.

Authorized licensed use limited to: IEEE Xplore. Downloaded on September 29,2024 at 14:05:39 UTC from IEEE Xplore. Restrictions apply.

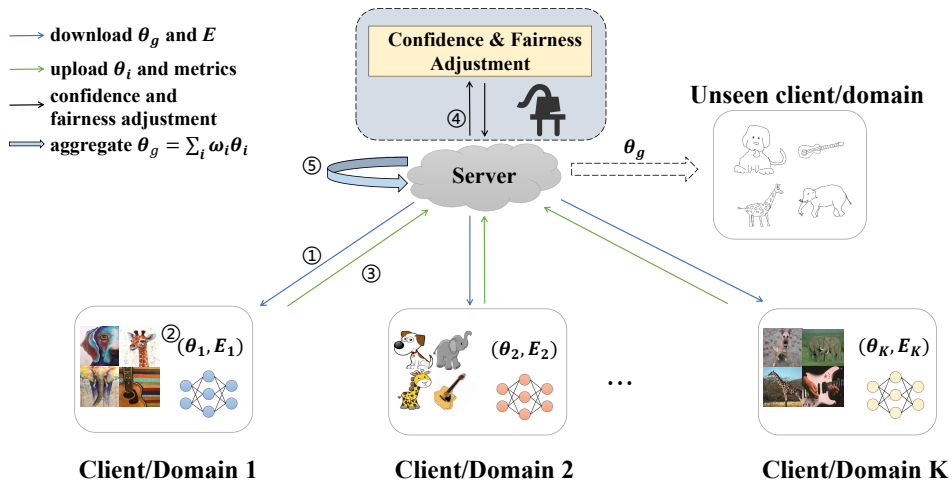

Fig. 1. Overview of our framework. The overall method consists of five steps: (1) downloading global model θ_g and local training epochs E, (2) local training, (3) uploading local model θ_i and metrics, (4) confidence and fairness adjustment in the server, and (5) aggregation.

3.2. Confidence Adjustment

In an ideal scenario where the local models trained on clients are optimal, the aggregation weights for each local model in the server should be uniform. However, due to limitations in the volume of data available on the client, such an assumption is unrealistic. Therefore, it becomes imperative to adjust the aggregation weights.

We denote the actual local model trained on the i-th domain/client as θ_i, and denote the difference between θ_i and the optimal local model θ_i^* for that domain as $\Delta\theta_i = \theta_i^* - \theta_i$. Clearly, as $\Delta\theta_i$ increases, the confidence of θ_i being the optimal model decreases, and consequently, the aggregation weight should decrease. Conversely, as $\Delta\theta_i$ decreases, the aggregation weight should increase.

Therefore, the methodology for perceiving $\Delta\theta_i$ becomes crucial. The fundamental factors influencing $\Delta\theta_i$ include both the quantity and quality of the samples. In general, the larger the quantity of samples N_i, the smaller $\Delta\theta_i$ becomes. Thus we consider the quantity of samples N_i as a metric for adjusting the aggregation weight. Additionally, the performance metrics (i.e. accuracy or loss value) on the local training and test set are also crucial, providing a tangible reflection of $\Delta\theta_i$, We choose accuracy on the test set (which can be formulated as Eq.(3)) and the difference in accuracy between training and test set (to perceive the degree of overfitting, which can be formulated as Eq.(5)) as indicators for perceiving $\Delta\theta_i$.

$$\mathrm{acc}_i^{test} = \frac{1}{N_i^{test}} \sum_{j=1}^{N_i^{test}} \mathbb{I}(f(x_i^j, \theta_i) == y_i^j), \quad (x_i^j, y_i^j) \in D_i^{test} \tag{3}$$

$$\mathrm{acc}_i^{train} = \frac{1}{N_i^{train}} \sum_{j=1}^{N_i^{train}} \mathbb{I}(f(x_i^j, \theta_i) == y_i^j), \quad (x_i^j, y_i^j) \in D_i^{train} \tag{4}$$

$$\Delta\mathrm{acc}_i = \mathrm{acc}_i^{test} - \mathrm{acc}_i^{train} \tag{5}$$

where N_i^{test} represents the number of samples in the test set of k-th domain/client, N_i^{train} represents the number of samples in the training set of k-th domain/client, D_i^{test} and D_i^{train} represents the test set and training set of k-th domain/client, and $\mathbb{I}$ is the indicator function.

In communication round r, the server sends the global model θ_g^r to each client. Subsequently, each client initializes its local model with θ_g^r as the initial weights and train local model θ_i^r using dataset D_i^{train}. After training is completed, each client tests θ_i^r on the local data, obtaining metrics $\mathrm{acc}_{i,r}^{test}$ and $\Delta\mathrm{acc}_{i,r}$. Then each client uploads $\{\theta_i^r, \mathrm{acc}_{i,r}^{test}, \Delta\mathrm{acc}_{i,r}, N_i\}$ to the server. Once the server receives the information sent by each client, firstly calculates the three factors $f_{i,1}, f_{i,2}, f_{i,3}$ that perceive $\Delta\theta_i$ for client i, which are shown in the following:

$$f_{i,1} = \log\left(\frac{\mathrm{acc}_{i,r}^{test} - min(\mathrm{acc}_{j,r}^{test})}{max(\mathrm{acc}_{j,r}^{test}) - min(\mathrm{acc}_{j,r}^{test})} + 1\right) \tag{6}$$

$$f_{i,2} = \log\left(\frac{\Delta\mathrm{acc}_{i,r} - min(\Delta\mathrm{acc}_{j,r})}{max(\Delta\mathrm{acc}_{j,r}) - min(\Delta\mathrm{acc}_{j,r})} + 1\right) \tag{7}$$

$$f_{i,3} = \frac{N_i}{\sum_{j=1}^{K} N_j} \tag{8}$$

Authorized licensed use limited to: IEEE Xplore. Downloaded on September 29,2024 at 14:05:39 UTC from IEEE Xplore. Restrictions apply.

where $min(\text{acc}_{j,r}^{test})$ is short for $min(\{\text{acc}_{j,r}^{test}\}_{1\leq j\leq K})$, and the same applies to $max(\text{acc}_{j,r}^{test})$, $min(\text{acc}_{j,r}^{test})$, $min(\Delta\text{acc}_{j,r})$ and $max(\Delta\text{acc}_{j,r})$. We normalize $acc_{i,r}^{test}$ and $\Delta\text{acc}_{i,r}$ to the range $[0,1]$, then take the logarithm (to smooth out the variation) as $f_{i,1}$ and $f_{i,2}$. $f_{i,3}$ is the ratio of the data volume on client i to the total data volume. Then new aggregation weight is calculated as follows:

$$f_i = \alpha_1 f_{i,1} + \alpha_2 f_{i,2} + \alpha_3 f_{i,3} \tag{9}$$

$$w_{i,r}^{'} = \frac{f_i}{\sum_j f_j} \tag{10}$$

$$w_{i,r} = \beta w_{i,r-1} + (1-\beta) w_{i,r}^{'} \tag{11}$$

where α_i, α_2 and α_3 represent the weights assigned to the three factors, respectively. We employ momentum updates to ensure the stability of the aggregation weight, and β represents the momentum of the update. In the implementation, we clip the $w_{i,r}^{'}$ greater than $\frac{1}{K}-t$ and smaller than $\frac{1}{K}+t$ to limit the magnitude of each adjustment process, t is a small positive value and set as 0.1.

3.3. Fairness Adjustment

Due to varying difficulty levels in model training across different domains, if a particular domain exhibits high training difficulty or has a small dataset, it may result in a reduction of its weight in the global model. This, in turn, leads to worse performance on that domain, creating a vicious cycle. Therefore, we design a fairness adjustment mechanism of local training epochs to ensure the fairness across domains.

The intuitive idea is to increase the local training epoch for domains that consistently perform poorly and decrease the local training epoch for domains that consistently perform well. The rule is shown in the following:

$$E_{i,r} = \begin{cases} E - \Delta E, & \{rk_{i,r'}\}_{r'=r-T}^{r} \leq R_{trigger} \\ E + \Delta E, & \{rk_{i,r'}\}_{r'=r-T}^{r} \geq K - R_{trigger} \\ E, & others \end{cases} \tag{12}$$

where $E_{i,r}$ represents the local training epoch of client i in the round r, $rk_{i,r}$ represents the ranking of $\text{acc}_{i,r}$ among all domains (in ascend order), $R_{trigger}$ and T represent the thresholds for performing fairness adjustment, e.i. when the ranking of acc_i^{test} of a particular domain remains consistently above/below $K-R_{trigger}/R_{trigger}$ for consecutive T rounds, the adjustment operation is triggered. $E-\Delta E$, E and $E+\Delta E$ are the low, medium and high categories of local training epoch, respectively, ΔE is the adjustment amount of local training epochs, which is set as 1. Once the adjustment operation is triggered, the count for consecutive rounds is reset.

Algorithm 1 FedAWA

Input: Global model $\theta_g = \theta^0$, K clients, the initial weights $\{w_{1,0}, w_{2,0}, ..., w_{K,0}\} = \{\frac{1}{K}, \frac{1}{K}, ..., \frac{1}{K}\}$. Round number R, the initial local training epochs $\{E_{1,1}, E_{2,1}, ..., E_{K,1}\} = \{E, E, ..., E\}, etc.$

Output: Global model θ_g after R iterations.

Server Operations:

for $r = 1, 2, ..., R-1$ **do**
  $\theta_i^{r-1} \leftarrow \theta_g$
  for clinet $i \in K$ **in parallel do**
    $\theta_i^r, \text{acc}_{i,r}^{test}, \Delta\text{acc}_{i,r}, N_i \leftarrow \text{ClientTrain}(\theta_i^{r-1}, E_{i,r})$
  end for
  calculate $f_{i,1}, f_{i,2}, f_{i,3}$ by $\{\text{acc}_{i,r}^{test}, \Delta\text{acc}_{i,r}, N_i\}_{i=1}^{K}$ $\triangleright Eq.(6)-(8)$
  calculate $w_{i,r}^{'}$ by $\{f_{i,1}, f_{i,2}, f_{i,3}\}_{i=1}^{K}$ $\triangleright Eq.(9)-(10)$
  $w_{i,r} = \beta w_{i,r-1} + (1-\beta) w_{i,r}^{'}$ $\triangleright Eq.(11)$
  update global model $\theta_g = \sum_{i=1}^{K} w_{i,r}\theta_i^{r-1}$
  calculate $\{rk_i\}_{i=1}^{K}$ by $\{\text{acc}_{i,r}^{test}\}_{i=1}^{K}$
  update local training epochs $E_{i,r+1}$ by $\{rk_{i,r'}\}_{r'=r-T}^{r}$ $\triangleright Eq.(12)$
end for
return θ_g

ClientTrain($\theta_i, E_{i,r}$)**:**

for $e = 1, 2, ..., E_{i,r}$ **do**
  for batch $\{x, y\} \in D_i^{train}$ **do**
    $\theta_i \leftarrow \text{update}(\theta_i, \{x, y\})$ $\triangleright$ Gradient descent
  end for
end for
calculate $\text{acc}_{i,r}^{train}$ by θ_i and D_i^{train} $\triangleright Eq.(3)$
calculate $\text{acc}_{i,r}^{test}$ by θ_i and D_i^{test} $\triangleright Eq.(4)$
$\Delta\text{acc}_{i,r} = \text{acc}_{i,r}^{train} - \text{acc}_{i,r}^{test}$ $\triangleright Eq.(5)$
return $\theta_i^r, \text{acc}_{i,r}^{test}, \Delta\text{acc}_{i,r}, N_i = |D_i|$

The workflow of FedAWA is illustrated in Algorithm 1. Confidence adjustment and fairness adjustment can serve as a plug-and-play plugin adaptable to most federated learning algorithms, such as FedAvg and so on.

4. EXPERIMENT

4.1. Datasets

We evaluate our proposed method on two widely used datasets in domain generalization, namely **PACS** [23] and **Office-Home** [24].

PACS is an image recognition benchmark including 9,991 images, it consists of four domains, namely Photo (1,670 images), Art Painting (2,048 images), Cartoon (2,344 images) and Sketch (3,929 images). Each domain contains 7 categories.

Authorized licensed use limited to: IEEE Xplore. Downloaded on September 29,2024 at 14:05:39 UTC from IEEE Xplore. Restrictions apply.

Office-Home is another image recognition benchmark including 15,588 images, it consists of four domains too, namely Art (2,427 images), Clipart (4,365 images), Product (4,439 images) and Real World (4,357 images). Each domain contains 65 categories.

4.2. Baselines

We select the following baselines:

- **FedAvg** (AISTATS 17) [15] is one of the most famous methods in federated learning, and it also acts as a strong baseline in FDG.
- **FedProx** (MLsys 20) [16] regularizes the local model training with a proximal term in the model objective to achieve faster convergence and alleviate the impact of data heterogeneity.
- **FedSR** (NeurIPS 22) [5] is one of most recent work in FDG, it enforces an L2-norm regularizer on the representation and a conditional mutual information regularizer to encourage the model to only learn essential information.
- **FedDG-GA** (CVPR 23) [6] is one of most recent work in FDG too, it proposes generalization adjustment to calibrate the aggregation weights dynamically.

We compare each algorithm (i.e. FedAvg, FedProx and FedSR) with/without FedAWA. Additionally, we will compare these algorithms with FedDG-GA due to FedDG-GA can also serve as a plugin.

4.3. Experiment Settings

We take each domain as a single client and perform "leave-one-domain-out" experiments on PACS and Office-Home datasets. In other words, we select one client as target test domain and others as training domains. For two datasets, we use 90% of available data as training data, 10% as test data in training domains, and 100% of available data as test data in the target domain.

We choose ResNet18 [25] pre-trained on ImageNet as our backbone for PACS and Office-Home, which is the most common choice of backbone in existing works, we replace the last fully connected layer of the backbone with a linear layer of dimension 512 to form the representation network and use a single layer to map from the representation to the output, which is to ensure identical network structures in comparison with FedSR. For PACS, We train the model using SGD optimizer with $1e^{-2}$ learning rate for 50 communication rounds. For Office-Home, We train the model using SGD optimizer with $1e^{-3}$ learning rate for 30 communication rounds. For two datasets, the batch size is set as 64, the momentum of update β is set as 0.9, the thresholds for performing fairness adjustment T and $R_{trigger}$ are set as 4 and 1, respectively. And the weights of three factors are set as $\alpha_1 = \alpha_2 = 1.0$, $\alpha_3 = 0.5$, the local training epoch E is set as 1. During the training, we resize our data to 224×224 pixels and normalize our data using the ImageNet channel means and standard deviations. The hyperparameters in the baselines are kept consistent with their default values from the open-source code.

Table 1. Accuracy comparison of image recognition on the PACS. "+GA" represents using FedDG-GA, "+AWA" represents using FedAWA. Reported numbers are from 3 runs.

Method	A	C	P	S	**Avg.**
FedAvg	78.04	79.22	91.62	81.17	82.51
FedAvg+GA	82.06	75.20	93.77	80.73	82.94
FedAvg+AWA	82.23	78.74	93.69	81.30	**83.99**
FedProx	77.51	79.05	91.80	80.52	82.22
FedProx+GA	82.21	74.32	94.17	81.32	83.00
FedProx+AWA	81.74	78.67	93.25	81.06	**83.70**
FedSR	79.87	79.48	92.56	81.02	83.23
FedSR+GA	82.85	77.74	94.71	81.11	84.10
FedSR+AWA	82.99	79.56	94.17	81.35	**84.52**
Avg.	78.47	79.25	91.99	80.90	82.65
Avg. (+GA)	82.37	75.75	94.22	81.05	83.35
Avg. (+AWA)	82.32	79.00	93.70	81.24	**84.07**

Table 2. Accuracy comparison of image recognition on the Office-Home. "+GA" represents using FedDG-GA, "+AWA" represents using FedAWA. Reported numbers are from 3 runs.

Method	A	C	P	R	**Avg.**
FedAvg	56.49	47.99	71.71	73.13	62.33
FedAvg+GA	56.63	49.13	69.59	73.15	62.13
FedAvg+AWA	57.24	48.37	71.34	73.84	**62.70**
FedProx	56.26	47.97	71.68	73.15	62.27
FedProx+GA	56.49	48.94	68.93	72.97	61.83
FedProx+AWA	57.04	48.35	71.38	73.59	**62.59**
FedSR	57.57	48.47	71.59	73.71	62.84
FedSR+GA	57.00	48.87	69.37	73.75	62.25
FedSR+AWA	58.25	48.99	71.31	74.39	**63.24**
Avg.	56.77	48.14	71.66	73.33	62.48
Avg. (+GA)	56.71	48.98	69.30	73.29	62.07
Avg. (+AWA)	57.51	48.57	71.34	73.94	**62.84**

The framework is implemented with PyTorch and is trained on a single NVIDIA RTX3090 GPU.

Authorized licensed use limited to: IEEE Xplore. Downloaded on September 29,2024 at 14:05:39 UTC from IEEE Xplore. Restrictions apply.

Table 3. Ablation study of the modules of our proposed method on the PACS. "AM" represents the use of arithmetic mean for aggregation weights, "CA" represents the use of confidence adjustment of aggregation weights, and "FA" represents the use of fairness adjustment of local training epochs. Reported numbers are from 3 runs.

AM	**CA**	**FA**	A	C	P	R	**Avg.**
			78.04	79.22	91.62	81.17	82.51
✓			81.15	77.66	93.83	81.86	83.63
✓	✓		82.28	76.55	93.25	81.46	83.39
✓		✓	80.71	79.96	93.29	81.61	83.89
✓	✓	✓	82.23	78.74	93.69	81.30	**83.99**

4.4. Comparison Results

We report the performance on PACS in Table 1. It can be observed that FedAWA contributes to an improvement in the performance compared to different algorithms, and our method also exhibits some improvement compared to different algorithms with FedDG-GA. On each domain, FedAWA does not result in significant performance degradation, while FedDG-GA leads to a significant performance degradation (-3.50%) on domain **C**artoon, which indicates that FedAWA is more robust than FedDG-GA. Of course, due to randomness, we do not particularly focus on the performance of a particular domain, but rather on the average performance of multiple experiments on different domains.

The performance on Office-Home is reported in Table 2. We believe that, due to the relatively balanced data volumes across domains in this dataset, the gap between arithmetic mean and weighted average based on data volume is little, resulting in a modest improvement. However, FedAWA still exhibits some improvement. FedAWA does not exhibit significant negative optimization on any domain while FedDG-GA brings a substantial performance decline (-2.36%) on domain **P**roduct, which is similar to the findings observed on PACS and once again proves the robustness of our proposed method. WW

4.5. Ablation study

We provide the ablation study on different modules of our method in Table 3. According to the results, we observe that the global model shows the best performance when equipped with AM, CA and FA. Besides, we observe that there is also a significant improvement in performance when only equipped with AM. When only equipped with CA, there is a decrease in performance compared to only equipped with AM, which we believe is due to the reason mentioned in Sec. 3.3, but compared to the vanilla FedAvg, there is still an improvement. When equipped with CA and FA together, the performance is better than equipped with A or B alone, which indicates that the two modules can complement each other's advantages.

5. CONCLUSION

In this paper, we proposed a new method to solve the DG problem under the FL setting. Our method reexamines the aggregation weights in FDG and replaces the weighted average with arithmetic mean of aggregation weights. We employ confidence adjustment to further ensure the rationality of the aggregation weights, and employ fairness adjustment of local training epochs to ensure that no client gradually dominates or gets abandoned. Moreover, our method can serve as a plug-and-play plugin to be used in conjunction with traditional federated learning algorithms and it brings performance improvement in FDG.

6. ACKNOWLEDGEMENT

This work was supported by NSFC under Grant No. 61936011, Beijing Key Lab of Networked Multimedia.

7. REFERENCES

[1] Jan Philipp Albrecht, "How the GDPR will change the world," *Eur. Data Prot. L. Rev.*, vol. 2, pp. 287, 2016.

[2] Qinbin Li, Zeyi Wen, Zhaomin Wu, Sixu Hu, Naibo Wang, Yuan Li, Xu Liu, and Bingsheng He, "A survey on federated learning systems: Vision, hype and reality for data privacy and protection," *IEEE Transactions on Knowledge and Data Engineering*, 2021.

[3] Liling Zhang, Xinyu Lei, Yichun Shi, Hongyu Huang, and Chao Chen, "Federated learning for iot devices with domain generalization," *IEEE Internet of Things Journal*, 2023.

[4] Quande Liu, Cheng Chen, Jing Qin, Qi Dou, and Pheng-Ann Heng, "Feddg: Federated domain generalization on medical image segmentation via episodic learning in continuous frequency space," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 1013–1023.

[5] A Tuan Nguyen, Philip Torr, and Ser Nam Lim, "Fedsr: A simple and effective domain generalization method for federated learning," *Advances in Neural Information Processing Systems*, vol. 35, pp. 38831–38843, 2022.

[6] Ruipeng Zhang, Qinwei Xu, Jiangchao Yao, Ya Zhang, Qi Tian, and Yanfeng Wang, "Federated domain generalization with generalization adjustment," in *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 3954–3963.

Authorized licensed use limited to: IEEE Xplore. Downloaded on September 29,2024 at 14:05:39 UTC from IEEE Xplore. Restrictions apply.

[7] Kaiyang Zhou, Ziwei Liu, Yu Qiao, Tao Xiang, and Chen Change Loy, "Domain generalization: A survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2022.

[8] Jindong Wang, Cuiling Lan, Chang Liu, Yidong Ouyang, Tao Qin, Wang Lu, Yiqiang Chen, Wenjun Zeng, and Philip Yu, "Generalizing to unseen domains: A survey on domain generalization," *IEEE Transactions on Knowledge and Data Engineering*, 2022.

[9] Shiv Shankar, Vihari Piratla, Soumen Chakrabarti, Siddhartha Chaudhuri, Preethi Jyothi, and Sunita Sarawagi, "Generalizing across domains via cross-gradient training," *arXiv preprint arXiv:1804.10745*, 2018.

[10] Xiangyu Yue, Yang Zhang, Sicheng Zhao, Alberto Sangiovanni-Vincentelli, Kurt Keutzer, and Boqing Gong, "Domain randomization and pyramid consistency: Simulation-to-real generalization without accessing target domain data," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 2100–2110.

[11] Aayush Prakash, Shaad Boochoon, Mark Brophy, David Acuna, Eric Cameracci, Gavriel State, Omer Shapira, and Stan Birchfield, "Structured domain randomization: Bridging the reality gap by context-aware synthetic data," in *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 7249–7255.

[12] Maximilian Ilse, Jakub M Tomczak, Christos Louizos, and Max Welling, "Diva: Domain invariant variational autoencoders," in *Medical Imaging with Deep Learning*. PMLR, 2020, pp. 322–348.

[13] Haoliang Li, Sinno Jialin Pan, Shiqi Wang, and Alex C Kot, "Domain generalization with adversarial feature learning," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 5400–5409.

[14] Massimiliano Mancini, Samuel Rota Bulo, Barbara Caputo, and Elisa Ricci, "Best sources forward: domain generalization through source-specific nets," in *2018 25th IEEE international conference on image processing (ICIP)*. IEEE, 2018, pp. 1353–1357.

[15] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Artificial intelligence and statistics*. PMLR, 2017, pp. 1273–1282.

[16] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith, "Federated optimization in heterogeneous networks," *Proceedings of Machine learning and systems*, vol. 2, pp. 429–450, 2020.

[17] Xin Yao, Tianchi Huang, Chenglei Wu, Ruixiao Zhang, and Lifeng Sun, "Towards faster and better federated learning: A feature fusion approach," in *2019 IEEE International Conference on Image Processing (ICIP)*. IEEE, 2019, pp. 175–179.

[18] Xin Yao, Tianchi Huang, Chenglei Wu, Rui-Xiao Zhang, and Lifeng Sun, "Federated learning with additional mechanisms on clients to reduce communication costs," *arXiv preprint arXiv:1908.05891*, 2019.

[19] Xin Yao, Chaofeng Huang, and Lifeng Sun, "Two-stream federated learning: Reduce the communication costs," in *2018 IEEE Visual Communications and Image Processing (VCIP)*, 2018, pp. 1–4.

[20] Durmus Alp Emre Acar, Yue Zhao, Ramon Matas Navarro, Matthew Mattina, Paul N Whatmough, and Venkatesh Saligrama, "Federated learning based on dynamic regularization," *arXiv preprint arXiv:2111.04263*, 2021.

[21] Jianyu Wang, Qinghua Liu, Hao Liang, Gauri Joshi, and H Vincent Poor, "Tackling the objective inconsistency problem in heterogeneous federated optimization," *Advances in neural information processing systems*, vol. 33, pp. 7611–7623, 2020.

[22] Junming Chen, Meirui Jiang, Qi Dou, and Qifeng Chen, "Federated domain generalization for image recognition via cross-client style transfer," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2023, pp. 361–370.

[23] Da Li, Yongxin Yang, Yi-Zhe Song, and Timothy M Hospedales, "Deeper, broader and artier domain generalization," in *Proceedings of the IEEE international conference on computer vision*, 2017, pp. 5542–5550.

[24] Hemanth Venkateswara, Jose Eusebio, Shayok Chakraborty, and Sethuraman Panchanathan, "Deep hashing network for unsupervised domain adaptation," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 5018–5027.

[25] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.

Authorized licensed use limited to: IEEE Xplore. Downloaded on September 29,2024 at 14:05:39 UTC from IEEE Xplore. Restrictions apply.